

How to Never Hit Your Token Limits in Claude Code

Claude Code counts tokens, not messages. Every file, folder, and system prompt eats your window. Here are 8 habits that keep every session long.

5 KEY HABITS

1 `/compact` at 50%

Run `/compact` at the halfway mark. Claude summarises the conversation and resets the message history while keeping the important context. If you wait until 95%, you are compacting a window already full of noise. The summary will be weaker and Claude will carry forward degraded context for the rest of the session. Compact early and your second half runs as cleanly as the first.

2 `/clear` Between Tasks

Every message in your session gets re-read by Claude on every single turn. That means a long back-and-forth from your last task is burning tokens on your next one, even if it is completely unrelated. When a task is done, type `/clear` and start fresh. Treat each task like a new session and your token budget stretches across far more work.

3 Read Files, Not Folders

When you point Claude at a folder it reads the entire directory tree before doing anything. Name the exact file you need instead. This one habit alone can cut your token spend per turn by 90% on file-heavy projects. If you are not sure of the exact filename, ask Claude to list files first, then read only the one you need.

A single file costs ~800 tokens. A folder tree costs ~12,000.

4 `/plan` Before You Build

`/plan` is a read-only mode. Claude maps out the work without writing a single line of code. It costs almost nothing compared to a full build run. One failed build, one round of debugging, and one fix cycle can easily burn more tokens than an entire planning session. Spend five minutes in `/plan` and your build runs first time far more often.

5

/context to Audit

Run /context at any point to see a full breakdown of what is inside your context window. It shows your system prompt, every MCP tool loaded, memory files, and the message history, all itemised by token count. Most people are surprised by how much their system prompt and MCP tools cost before they have typed a single message. Use /context to spot what to trim.

MATCH /EFFORT TO THE TASK

low / medium

Quick edits, formatting, simple refactors, and boilerplate that does not need deep reasoning. Use this for renaming variables, updating comments, or restructuring imports.

Uses Sonnet. Fastest and cheapest.

high

Real coding, debugging, multi-step workflows, and anything requiring logic across multiple files. This is the right setting for the bulk of your working day.

Default. Uses Sonnet.

xhigh / max

Complex architecture decisions, hard bugs with unclear causes, and anything where a wrong call is expensive to undo. Reserve this for problems that genuinely need it.

Uses Opus. Burns tokens fastest.

Sonnet executes. Opus strategises. Match the effort to the task.

SET ONCE, FORGET

6

CLAUDE.md

Write a CLAUDE.md

A CLAUDE.md file sits at your project root and Claude reads it automatically at the start of every session. Put your stack, your coding rules, your naming conventions, and your tone in one place. You stop re-pasting the same instructions every time you open a new session. Claude starts each task already knowing your preferences, your tech choices, and what to avoid. One file, zero repeated setup.

7

Skills

Build Skills

A skill is a saved prompt file stored in your .claude/skills folder. When Claude needs it, it loads the skill automatically instead of you pasting the same instructions again. If you have a prompt you use more than twice, turn it into a skill. Post writer, carousel builder, email drafter, code reviewer. Each one saves context on every run and keeps your main thread focused on the actual task.

8

Subagents

Use Subagents

When you need to read a large codebase, research a topic, or process a batch of files, send that work to a subagent. The subagent runs in its own separate context window. It does the heavy reading, processes the information, and hands back a clean summary. Your main thread never sees the raw volume. You get the answer without spending your own context budget on the legwork.

Get more free AI guides at charliehills.substack.com